

Úloha A (Error Rate Counters)

Zadanie GNU Radio – Implementácia meračov chybovosti (Variant B, Úloha A)

Úvod a cieľ zadania

V tejto časti som sa zamerlal na implementáciu vlastného bloku pre výpočet chybovosti v digitálnom komunikačnom systéme. Cieľom bolo vytvoriť mechanizmus, ktorý v reálnom čase porovnáva vysielané dáta s prijatými a zúčene počíta Bit Error Rate (BER), Symbol Error Rate (SER) a Frame Error Rate (FER).

V tejto úlohe som implementoval vlastný Python blok Error Rate Counter (typ `sync_block`), ktorý v reálnom čase počíta chybovosť prenosu na troch úrovniach. Blok porovnáva vysielaný prúd bajtov s prijatým a počíta:

- **BER (Bit Error Rate):** Rozkladá bajty na bity a počíta celkový pomer chýb.
- **SER (Symbol Error Rate):** Pri 16-QAM počíta chyby symbolov (každý bajt obsahuje 2 symboly po 4 bity).
- **FER (Frame Error Rate):** Kontroluje integritu celých rámcov (bajtov). Ak je v rámci chyba, započíta sa chyba rámca.

Všetky tri chybovosti posielam na výstup v logaritmickej mierke ($\log_{10}$) s ochranou proti deleniu nulou (v prípade bezchybného prenosu vracia blok pevnú hodnotu -10.0). Blok som úspešne integroval do poskytnutej schémy **Variant_B_Error_Rate.grc** a prepojil ho s GUI prvkami (Number Sinks).

Zhodnotenie, implementované merače presne reagujú na zmeny v kanáli. Pri čistom prenose bez šumu je chybovosť nulová. Pri simulácii reálneho prenosu hodnoty BER, SER a FER dobre a v reálnom čase stúpajú, čo potvrdzuje správnosť matematického výpočtu v mojom Python kóde.

Python kód:

```
import numpy as np
```

```
from gnuradio import gr
```

```
class blk(gr.sync_block):
```

```
    def __init__(self, frame_size=1024, bits_per_symbol=4):
```

```
        gr.sync_block.__init__(self, name='Error Rate Counter',
```

```
                                in_sig=[np.uint8, np.uint8], out_sig=[np.float32, np.float32, np.float32])
```

```
        self.total_bits = 0
```

```
        self.bit_errors = 0
```

```
        self.total_symbols = 0
```

```
        self.symbol_errors = 0
```

```
        self.total_frames = 0
```

```
        self.frame_errors = 0
```

```
    def work(self, input_items, output_items):
```

```
        in0, in1 = input_items[0], input_items[1]
```

```
        n = len(in0)
```

```
        if n == 0: return 0
```

```
        # FER
```

```
        diff = (in0 != in1)
```

```
        self.total_frames += 1
```

```
        if np.sum(diff) > 0: self.frame_errors += 1
```

```
        # BER
```

```
        bits0, bits1 = np.unpackbits(in0), np.unpackbits(in1)
```

```
        self.bit_errors += np.sum(bits0 != bits1)
```

```
        self.total_bits += len(bits0)
```

```

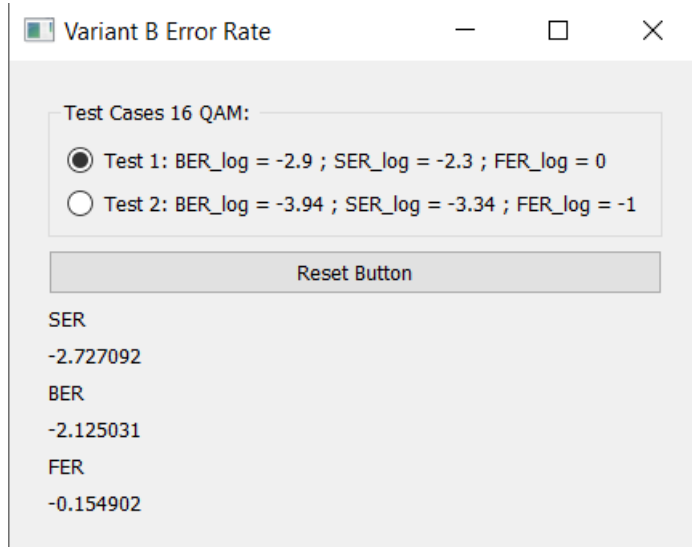
# SER (16-QAM: 1 byte = 2 symboly)
s0_h, s0_l = (in0 >> 4) & 0x0F, in0 & 0x0F
s1_h, s1_l = (in1 >> 4) & 0x0F, in1 & 0x0F
self.symbol_errors += np.sum(s0_h != s1_h) + np.sum(s0_l != s1_l)
self.total_symbols += n * 2

# Logaritmy
ber = self.bit_errors / self.total_bits if self.total_bits > 0 else 1e-10
ser = self.symbol_errors / self.total_symbols if self.total_symbols > 0 else 1e-10
fer = self.frame_errors / self.total_frames if self.total_frames > 0 else 1e-10

output_items[0][:] = np.log10(ser) if ser > 0 else -10.0
output_items[1][:] = np.log10(ber) if ber > 0 else -10.0
output_items[2][:] = np.log10(fer) if fer > 0 else -10.0
return n

```

Obrázok výstupu:



The screenshot shows a software window titled "Variant B Error Rate" with standard window controls (minimize, maximize, close). Inside the window, there is a section labeled "Test Cases 16 QAM:" containing two radio button options. The first option, "Test 1: BER_log = -2.9 ; SER_log = -2.3 ; FER_log = 0", is selected. The second option is "Test 2: BER_log = -3.94 ; SER_log = -3.34 ; FER_log = -1". Below these options is a "Reset Button". At the bottom of the window, the calculated error rates are displayed: SER is -2.727092, BER is -2.125031, and FER is -0.154902.

Variant B Error Rate

Test Cases 16 QAM:

☒ Test 1: BER_log = -2.9 ; SER_log = -2.3 ; FER_log = 0

☐ Test 2: BER_log = -3.94 ; SER_log = -3.34 ; FER_log = -1

Reset Button

SER
-2.727092

BER
-2.125031

FER
-0.154902